

# pytch v2: A Real-Time Monitoring Tool For Polyphonic Singing Performances

Sebastian Rosenzweig<sup>1\*</sup>, Marius Kriegerowski<sup>2\*</sup>, and Frank Scherbaum<sup>3</sup>

<sup>1</sup> Independent Researcher, Barcelona, Spain <sup>2</sup> Independent Researcher, Berlin, Germany <sup>3</sup> University of Potsdam, Potsdam, Germany ¶ Corresponding author \* These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Fabian-Robert Stöter](#)

## Reviewers:

- [@aRI0U](#)
- [@hadware](#)

Submitted: 01 July 2025

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Polyphonic singing is one of the most widespread forms of music-making. During a performance, singers must constantly adjust their pitch to stay in tune with one another — a complex skill that requires extensive practice. Research has shown that pitch monitoring tools can assist singers in fine-tuning their intonation during a performance (Berglin et al., 2022). Specifically, real-time visualizations of the fundamental frequency (F0), which represents the pitch of the singing voice, help singers assess their pitch relative to a fixed reference or other voices. To support the monitoring of polyphonic singing performances, we developed pytch, an interactive Python tool with a graphical user interface (GUI) designed to record, process, and visualize multiple voices in real time. The GUI displays vocal spectra and estimated F0 trajectories for all singers, as well as the harmonic intervals between them. Additionally, users can adjust visual and algorithmic parameters interactively to accommodate different input devices, microphone signals, singing styles, and use cases. Our tool builds upon a late-breaking demo in (Kriegerowski & Scherbaum, 2017), which we refer to as version 1. Since its initial release, the tool has been significantly extended with a new real-time graphics engine, a modular audio processing backend that facilitates the integration of additional algorithms, and improved support for a wider range of platforms and recording hardware, which we refer to as version 2. The applications of pytch range from research in the field of computational musicology to pedagogical contexts focused on intonation and harmonic listening.

## State of the Field

Software that assesses the pitch of a singing voice in real time is best known from Karaoke singing applications, such as Let's Sing<sup>1</sup>, Rock Band<sup>2</sup>, or Cantamus<sup>3</sup>. These tools typically compare the singer's pitch to a score reference to judge whether notes are 'correct' or 'incorrect'. However, such applications face several limitations when applied to polyphonic or group singing contexts. Most notably, many Karaoke systems can only process one or two singing voices at a time, which is problematic for monitoring group performances. Additionally, software that relies on a score as a reference poses challenges for a cappella performances, where singers may drift together in pitch over time while maintaining relative harmony, or in orally-transmitted traditions that may lack a formal score altogether. Finally, existing open-source research software for singing voice processing, like Praat (Boersma, 2001), Sonic Visualiser (Cannam et al., 2010), and Tarsos (Six et al., 2013), lack real-time feedback, preventing an effective feedback loop between singers and their tool.

Taken together, these limitations highlight a gap in current approaches: there is no tool that

<sup>1</sup>[https://en.wikipedia.org/wiki/Let%27s\\_Sing](https://en.wikipedia.org/wiki/Let%27s_Sing)

<sup>2</sup>[https://en.wikipedia.org/wiki/Rock\\_Band\\_4](https://en.wikipedia.org/wiki/Rock_Band_4)

<sup>3</sup><https://cantamus.app>

39 supports real-time pitch monitoring for multiple voices in a way that reflects the needs of  
40 ensemble singing practices.

## 41 Statement of Need

42 To address the lack of available software, we developed pytch. Our tool enables singers  
43 and conductors to monitor and train harmonic interval singing in real time — a skill that is  
44 essential in many vocal traditions. This includes not only polyphonic genres such as traditional  
45 Georgian vocal music (Scherbaum et al., 2019) or Barbershop singing (Hagerman & Sundberg,  
46 1980), where precise tuning between voices is stylistically central, but also the practice of  
47 non-tempered tuning systems found in various oral traditions. Unlike many existing tools,  
48 pytch neither requires a musical score nor is it tied to a specific musical tuning system. The  
49 vocal spectra can help singers fine-tune the expression of formant frequencies, while melodic  
50 and harmonic issues become visible through F0 trajectories and harmonic intervals.

## 51 Research Impact Statement

52 Over its seven years of development to now, pytch has been tested and iterated through use  
53 in ensemble rehearsals, singing workshops, and ethnomusicological field research. For example,  
54 the tool was used to analyze the Sardinian singing style of the “Quintina” in which four singers  
55 manage to fuse the high frequency partials of their voices in such a way that an apparent fifth  
56 voice appears (see demo video<sup>4</sup>). Furthermore, pytch has been used during a field expedition  
57 to Georgia, where ethnomusicologists recorded and analyzed traditional vocal music which  
58 belongs to the UNESCO intangible world cultural heritage (see demo video<sup>5</sup>).

59 In addition to its use in musicological research, pytch also provides a platform for music  
60 information retrieval (MIR) research on real-time audio processing. Working with real-time  
61 data introduces challenges such as a limited audio context for analysis and strict timing  
62 constraints to ensure low-latency processing. Researchers can use pytch to develop, test, and  
63 compare algorithms for F0 estimation and other music information retrieval tasks (Goto, 2004;  
64 Meier et al., 2024; Stefani & Turchet, 2022). A notable example is the rtswipe (Meier et  
65 al., 2025) library, which was inspired by pytch use cases and integrated after its initial release.  
66 Using its implementation significantly increased the speed and accuracy of F0 estimates in  
67 pytch; consequently, it was selected as the default algorithm in our tool from version 2.3  
68 onward.

## 69 Multitrack Singing Recordings

70 To obtain accurate monitoring results with pytch, each singer should be recorded with an  
71 individual microphone. We recommend using handheld dynamic or headset microphones and  
72 positioning singers with sufficient distance to reduce cross-talk. A practical alternative is the  
73 use of contact microphones (e.g., throat microphones), which capture vocal fold vibrations  
74 directly from the skin, are largely immune to interference from other singers, and have been  
75 successfully used in prior ensemble recordings (Scherbaum, 2016). While there is no strict  
76 limit on the number of input channels, we recommend up to four singers to maintain chart  
77 readability.

78 In addition to live monitoring, pytch can also be used to analyze pre-recorded multitrack  
79 singing performances. By playing back individual vocal tracks in a digital audio workstation

<sup>4</sup><https://www.uni-potsdam.de/de/soundscapelab/computational-ethnomusicology/the-benefit-of-body-vibration-recordings/real-time-analysis-of-larynx-microphone-recordings>

<sup>5</sup><https://youtu.be/LPt83Wqf2e4>

(DAW) and using virtual audio routing tools such as Loopback<sup>6</sup> (macOS) or BlackHole<sup>7</sup>, these tracks can be streamed into pytch as if they were live microphone inputs. This setup, which was also used in the demo video<sup>8</sup>, allows users to benefit from pytch's real-time visualization and analysis features during evaluation of rehearsals, performances, or field recordings.

## Software Design

pytch is implemented in Python, the predominant language for research software in music computing, which facilitates contributions from other researchers. The pytch codebase systematically separates GUI code from audio processing code to ease maintenance.

## Audio Processing

The real-time audio processing pipeline implemented in the file `audio.py` is the heart of pytch and consists of two main stages: recording and analysis. The recording stage captures multichannel audio waveforms from the soundcard or an external audio interface using the `sounddevice` library<sup>9</sup>. The library is based on PortAudio<sup>10</sup> and supports a wide range of operating systems, audio devices, and sampling rates. The recorded audio is received in chunks via a recording callback and fed into a ring buffer shared with the analysis process. When the buffer is sufficiently filled with audio chunks, the analysis process reads the recorded audio to compute several audio features.

For each channel, the analysis stage computes the audio level in dBFS, a time–frequency representation of the audio signal via the Short-Time Fourier Transform (see (Müller, 2021) for fundamentals of music processing), and an estimate of the F0 along with a confidence value. There are currently two F0 algorithms available: YIN (Cheveigné & Kawahara, 2002) from the `libf0` library (Rosenzweig et al., 2022), and a real-time version of SWIPE (Camacho & Harris, 2008) from the `rtswipe` library (Meier et al., 2025). YIN is a time-domain algorithm that computes the F0 based on a tweaked auto-correlation function. It is computationally efficient and well-suited for low-latency applications, but it tends to suffer from estimation errors, particularly confusions with higher harmonics such as the octave. SWIPE is a frequency-domain algorithm that computes the F0 by correlating pitch candidate kernels derived from a sawtooth waveform with the spectrum of an input signal. It is known to be more robust to noise as YIN.

The obtained F0 estimates, which are natively computed in the unit Hz, are converted to the unit cents using a user-specified reference frequency. Depending on the audio quality and vocal characteristics, F0 estimates may exhibit artifacts such as discontinuities or pitch slides, which can make the resulting trajectories difficult to interpret (Rosenzweig et al., 2019). Previous research has shown that using throat microphones can improve the isolation of individual voices in group singing contexts, resulting in cleaner signals and more accurate F0 estimates (Scherbaum, 2016). To further enhance interpretability, pytch includes several optional post-processing steps: a confidence threshold to discard estimates with low confidence score, a median filter to smooth the trajectories, and a gradient filter to suppress abrupt pitch slides. As a final step in the audio analysis, the harmonic intervals between the F0 trajectories are computed. Every audio feature is stored separately in a dedicated ring buffer. After processing, the pipeline sets a flag that notifies the GUI that new data is ready for visualization.

<sup>6</sup><https://rogueamoeba.com/loopback/>

<sup>7</sup><https://existential.audio/blackhole/>

<sup>8</sup><https://youtu.be/LPt83Wqf2e4>

<sup>9</sup><https://github.com/spatialaudio/python-sounddevice>

<sup>10</sup><https://www.portaudio.com>

## Graphical User Interface (GUI)

In this section, we provide a step-by-step explanation of the pytch GUI implemented in the file `gui.py`. Inspired by other scientific Python-based tools (Aubier, 2025; Pataky et al., 2019), we designed the pytch GUI using the open-source library `pyqtgraph`<sup>11</sup>.

Right after the program start, a startup menu opens in which the user is asked to specify the soundcard, input channels, sampling rate, and window size for processing (see Figure Figure 1). Furthermore, the user can choose to store the recorded audio and the F0 trajectories on disk.

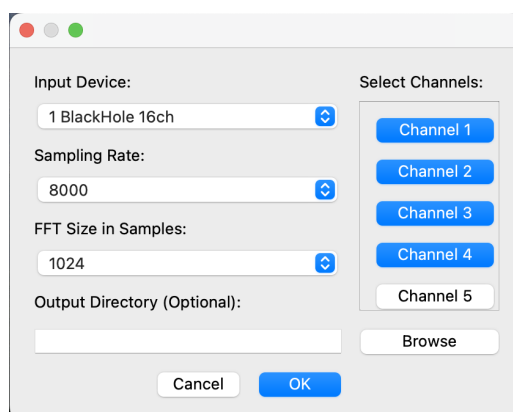


Figure 1: pytch startup menu.

These configuration choices are required to initialize the audio processing module and the main GUI, which is loaded when the user clicks “ok”. A screenshot of the main GUI which opens after successful initialization is shown in Figure Figure 2.

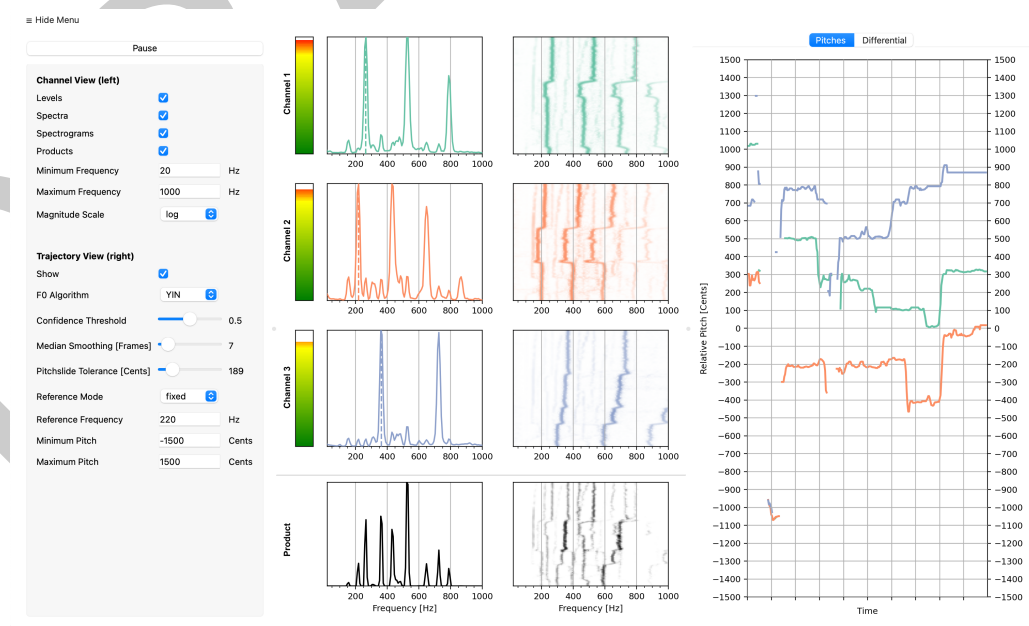


Figure 2: pytch GUI monitoring three singing voices.

The main GUI is organized into three horizontal sections. On the left, a control panel provides a start/stop button and allows users to adjust both the visual layout and algorithmic parameters.

<sup>11</sup><https://www.pyqtgraph.org>

133 The central section displays “channel views”—one for each input channel—color-coded for clarity.  
134 Each view includes a microphone level meter, a real-time spectrum display with a vertical line  
135 marking the current F0 estimate, and a scrolling spectrogram with a 5 second time context.  
136 Channels are listed from top to bottom in the order they were selected during setup. Optionally,  
137 the bottommost view can display a product signal from all channels.

138 The right section, referred to as the “trajectory view,” provides time-based visualizations of  
139 either the F0 trajectories (“pitches” tab) or the harmonic intervals between voices (“differential”  
140 tab) with a 10 second time context. Using the controls in the left-side menu, the user can  
141 select the F0 estimation algorithm and improve the real-time visualization by adjusting the  
142 confidence threshold, the median filter length for smoothing, and the tolerance of the gradient  
143 filter. F0 and interval trajectories can be displayed with respect to a fixed reference frequency  
144 or a dynamic one derived from a selected channel, the lowest, or highest detected voice. Axis  
145 limits for this section can also be manually set.

## 146 AI Usage Disclosure

147 The authors used the AI tool ChatGPT to assist with grammar, spelling, and language  
148 refinement in this article. In addition, ChatGPT was used to generate example code for PyQt6  
149 functions and classes. No code was copied verbatim from ChatGPT; all generated examples  
150 were reviewed, adapted, and integrated by the authors.

## 151 Acknowledgements

152 We would like to thank Lukas Dietz for his help with the implementation, Peter Meier,  
153 Sebastian Strahl, Simon Schwär, and Meinard Müller for the collaboration on the real-time  
154 SWIPE implementation, and all the singers who contributed to testing pytch during its  
155 development.

## 156 References

- 157 Aubier, T. (2025). *CoperniFUS: A flexible Python-based GUI for stereotaxic Focused*  
158 *UltraSound (FUS) experiment planning* (Version 0.1.2). Zenodo. <https://doi.org/10.5281/zenodo.15720551>  
159
- 160 Berglin, J., Pfordresher, P. Q., & Demorest, S. (2022). The effect of visual and auditory  
161 feedback on adult poor-pitch remediation. *Psychology of Music*, 50(4), 1077–1090.  
162 <https://doi.org/10.1177/03057356211026730>
- 163 Boersma, P. (2001). Praat, a system for doing phonetics by computer. *Glott International*,  
164 5(9/10), 341–345.
- 165 Camacho, A., & Harris, J. G. (2008). A sawtooth waveform inspired pitch estimator for speech  
166 and music. *The Journal of the Acoustical Society of America*, 124(3), 1638–1652.
- 167 Cannam, C., Landone, C., & Sandler, M. B. (2010). Sonic Visualiser: An open source  
168 application for viewing, analysing, and annotating music audio files. *Proceedings of the*  
169 *International Conference on Multimedia*, 1467–1468.
- 170 Cheveigné, A. de, & Kawahara, H. (2002). YIN, a fundamental frequency estimator for speech  
171 and music. *Journal of the Acoustical Society of America (JASA)*, 111(4), 1917–1930.
- 172 Goto, M. (2004). A real-time music-scene-description system: Predominant-F0 estimation for  
173 detecting melody and bass lines in real-world audio signals. *Speech Communication*, 43(4),  
174 311–329.

- 175 Hagerman, B., & Sundberg, J. (1980). *Fundamental frequency adjustment in barbershop*  
176 *singing*. Citeseer.
- 177 Kriegerowski, M., & Scherbaum, F. (2017). Pytch - simultane mehrkanalige audioanalyse von  
178 gesangstimmen. *Late-Breaking Demos of the Workshop: Musik Trifft Informatik at 47.*  
179 *Jahrestagung Der Gesellschaft Für Informatik*.
- 180 Meier, P., Chiu, C.-Y., & Müller, M. (2024). A real-time beat tracking system with zero  
181 latency and enhanced controllability. *Transactions of the International Society for Music*  
182 *Information Retrieval (TISMIR)*, 7(1), 213–227. <https://doi.org/10.5334/tismir.189>
- 183 Meier, P., Strahl, S., Schwär, S., Müller, M., & Balke, S. (2025). Pitch estimation in real time:  
184 Revisiting SWIPE with causal windowing. *Proceedings of the International Symposium on*  
185 *Computer Music Multidisciplinary Research (CMMR)*.
- 186 Müller, M. (2021). *Fundamentals of music processing – using python and jupyter notebooks*  
187 (2nd ed., pp. 1–495) [Monograph]. Springer Verlag. [https://doi.org/10.1007/978-3-030-](https://doi.org/10.1007/978-3-030-69808-9)  
188 [69808-9](https://doi.org/10.1007/978-3-030-69808-9)
- 189 Pataky, T., Naouma, H., & Donnelly, C. (2019). *mwarp1d: Manual one-dimensional data*  
190 *warping in Python and PyQt* (Version v0.2.0). Zenodo. [https://doi.org/10.5281/zenodo.](https://doi.org/10.5281/zenodo.3568447)  
191 [3568447](https://doi.org/10.5281/zenodo.3568447)
- 192 Rosenzweig, S., Scherbaum, F., & Müller, M. (2019). Detecting stable regions in frequency  
193 trajectories for tonal analysis of traditional Georgian vocal music. *Proceedings of the*  
194 *International Society for Music Information Retrieval Conference (ISMIR)*, 352–359. <https://doi.org/10.5281/zenodo.3527816>
- 196 Rosenzweig, S., Schwär, S., & Müller, M. (2022). libf0: A python library for fundamental  
197 frequency estimation. *Late Breaking Demos of the International Society for Music*  
198 *Information Retrieval Conference (ISMIR)*.
- 199 Scherbaum, F. (2016). On the benefit of larynx-microphone field recordings for the  
200 documentation and analysis of polyphonic vocal music. *Proceedings of the International*  
201 *Workshop Folk Music Analysis*, 80–87.
- 202 Scherbaum, F., Mzhavanadze, N., Rosenzweig, S., & Müller, M. (2019). Multi-media  
203 recordings of traditional Georgian vocal music for computational analysis. *Proceedings of*  
204 *the International Workshop on Folk Music Analysis (FMA)*, 1–6.
- 205 Six, J., Cornelis, O., & Leman, M. (2013). Tarsos, a modular platform for precise pitch analysis  
206 of Western and non-Western music. *Journal of New Music Research*, 42(2), 113–129.  
207 <https://doi.org/10.1080/09298215.2013.797999>
- 208 Stefani, D., & Turchet, L. (2022). On the challenges of embedded real-time music information  
209 retrieval. *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 3,  
210 177–184.